LECTURE SERIES · DATABASE SYSTEMS

Keys, Set Operations & Joins

Key Constraints and the Core Operators of Relational Algebra

Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

Keys · Set Operations · Joins · Worked Examples

IDENTIFYING A ROW UNIQUELY

Keys — Super, Candidate, Primary, Alternate

A key answers one question: which attribute set is enough to tell any two rows apart? The four are nested — **every primary key is a candidate key, and every candidate key is a super key**. The candidate keys left unchosen become alternate keys.

Super Key

Any attribute set that uniquely identifies a row — extra, unnecessary attributes are allowed.

`{StudentID}` · `{StudentID, Name}` · `{Email, Dept}`

Candidate Key

A minimal super key: drop any one attribute and uniqueness is lost.

`{StudentID}` `{NationalID}` `{Email}`

Primary Key

The one candidate key the designer picks. Must be unique and can never be NULL.

`PK = StudentID` (underlined in the ER diagram)

Alternate Key

Every candidate key not chosen as primary; usually enforced with a UNIQUE constraint.

`NationalID` · `Email` (a.k.a. secondary key)

KEYS THAT SPAN OR REFERENCE

Composite, Foreign & Partial Keys

Not every key is a single attribute of its own table. These three appear the moment relationships and weak entities enter the design.

Composite Key

Two or more attributes that are unique only in combination — the standard key of a junction table.

`Enroll( StudentID , CourseID )`

Foreign Key

An attribute referencing the primary key of another relation; it enforces referential integrity.

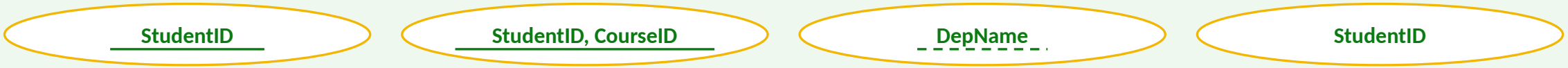
`Enroll.StudentID → Student.StudentID`

Partial Key

The discriminator of a weak entity — unique only when combined with the owner's primary key.

`Dependent( EmpID , DepName )`

HOW THESE APPEAR IN AN ER DIAGRAM



solid underline = primary key

both underlined = composite key

dashed underline = partial key

plain oval + relationship line = FK

ONE SCHEMA, EVERY KEY

Keys — Worked Example

Two relations from a university database. Read the key names off the data: uniqueness decides candidates, the designer decides the primary.

STUDENT

StudentID	NationalID	Email	Name	Dept
1901	1990112233	rakib@baust.ac	Rakib	CSE
1902	1990445566	alifa@baust.ac	Alifa	CSE
1903	1990778899	nadia@baust.ac	Nadia	EEE

ENROLL

StudentID	CourseID	Grade
1901	CSE-2205	A
1901	CSE-3102	A-
1902	CSE-2205	B+

Why a composite key?

A student takes many courses and a course has many students, so neither column alone is unique — together they are.

READING THE KEYS OFF THE TABLES

Super keys

`{StudentID}, {StudentID,Name}, {Email,Dept}`

Candidate keys

`{StudentID}, {NationalID}, {Email}`

Primary key

`StudentID`

Alternate keys

`NationalID, Email`

Composite key

`Enroll(StudentID, CourseID)`

Foreign key

`Enroll.StudentID → Student.StudentID`

Not a key

`Name, Dept — values repeat`

UNION-COMPATIBLE RELATIONS ONLY

Set Operations — Notation & Rules

Set operations treat relations as sets of tuples, so both inputs must be **union-compatible**: the same degree (number of attributes), with matching domains column by column. Duplicates are removed from the result.

Union

R ∪ S

Every tuple that appears in R or in S, counted once.

```
SELECT … FROM R
UNION
SELECT … FROM S;
```

Intersection

R ∩ S

Only the tuples that appear in both R and S.

```
SELECT … FROM R
INTERSECT
SELECT … FROM S;
```

Difference

R − S

Tuples of R with every tuple of S removed. R − S ≠ S − R.

```
SELECT … FROM R
EXCEPT
SELECT … FROM S;
```

Union compatibility is a schema rule, not a data rule: Student(SID, Name) and Teacher(TID, Name) can be unioned because both are (integer, string) — attribute names may differ, domains may not.

SAME TWO RELATIONS, THREE RESULTS

Set Operations — Worked Example

Two club membership lists, both of degree 2 over (SID, Name) — union-compatible, so all three operators apply.

CSE_CLUB (R)

SID	Name
1901	Rakib
1902	Alifa
1903	Nadia

SPORTS_CLUB (S)

SID	Name
1902	Alifa
1903	Nadia
1904	Imran

Rakib is only in CSE_Club, Imran is only in Sports_Club, and Alifa and Nadia are in both. **That overlap is what the three operators separate.**

Result sizes: union 4 · intersection 2 · difference 1 · S − R 1

CSE ∪ Sports

SID	Name
1901	Rakib
1902	Alifa
1903	Nadia
1904	Imran

duplicates removed — Alifa and Nadia appear once

CSE ∩ Sports

SID	Name
1902	Alifa
1903	Nadia

members of both clubs

CSE − Sports

SID	Name
1901	Rakib

in CSE_Club but not in Sports_Club

ONE IDEA, MANY FLAVOURS

The Join Family

Every join is a Cartesian product followed by a selection. The variants differ in the condition used and in what happens to rows that find no partner.

$$R \bowtie_\theta S \equiv \sigma_\theta(R \times S)$$

INNER JOINS

$\bowtie_\theta$ Theta Join
any condition θ: <, >, ≠, ≤

$\bowtie$ Equi Join
θ is equality; both columns kept

$\bowtie$ Natural Join
auto-match on shared attributes

unmatched rows are dropped

OUTER JOINS

$\overset{\frown}{\bowtie}$ Left Outer
all rows of the left relation kept

$\overset{\smile}{\bowtie}$ Right Outer
all rows of the right relation kept

$\overset{\frown\smile}{\bowtie}$ Full Outer
all rows of both relations kept

unmatched rows survive, padded with NULL

RELATED OPERATORS

× Cross Join
no condition at all — |R| × |S| rows

⋉ Semi Join
rows of R that have a partner in S

▷ Anti Join
rows of R that have no partner in S

same machinery, different purpose

KEEPING ONLY THE MATCHED ROWS

Inner Joins — Theta, Equi & Natural

Two relations sharing the attribute SID. Student 1902 has no enrolment and enrolment 1904 has no student — inner joins simply drop both.

STUDENT

SID	Name	CGPA
1901	Rakib	3.80
1902	Alifa	3.45
1903	Nadia	3.92

ENROLL

SID	CID
1901	CSE-2205
1903	CSE-3102
1904	CSE-4102

Student ⋈ Enroll

SID	Name	CGPA	CID
1901	Rakib	3.80	CSE-2205
1903	Nadia	3.92	CSE-3102

one SID column kept — 1902 and 1904 disappear

Theta Join — Student $\bowtie_{CGPA > 3.5}$ Enroll

Condition may be any comparison. Here every pair whose student CGPA exceeds 3.5 is kept, matched or not — 6 tuples of the 9 in the product.

Equi Join — Student $\bowtie_{Student.SID = Enroll.SID}$ Enroll

Theta join whose condition is equality. Same two rows as below, but SID appears twice — once from Student, once from Enroll.

Natural Join — Student ⋈ Enroll

Equality on every attribute that shares a name, with the duplicate column removed automatically. No condition is written.

KEEPING THE UNMATCHED ROWS TOO

Outer Joins — Left, Right & Full

Same Student and Enroll relations. An outer join keeps the rows an inner join would discard and pads the missing attributes with **NULL**.

Left Outer

Every Student row survives.

Student ⟕ Enroll

SID	Name	CGPA	CID
1901	Rakib	3.80	CSE-2205
1903	Nadia	3.92	CSE-3102
1902	Alifa	3.45	*NULL*

Right Outer

Every Enroll row survives.

Student ⟖ Enroll

SID	Name	CGPA	CID
1901	Rakib	3.80	CSE-2205
1903	Nadia	3.92	CSE-3102
1904	*NULL*	*NULL*	CSE-4102

Full Outer

Both unmatched rows survive.

Student ⟗ Enroll

SID	Name	CGPA	CID
1901	Rakib	3.80	CSE-2205
1903	Nadia	3.92	CSE-3102
1902	Alifa	3.45	*NULL*
1904	*NULL*	*NULL*	CSE-4102

NULL means “no matching row” — not zero, not an empty string. Counting or averaging a NULL-padded column in SQL silently skips those rows, so an outer join changes your aggregates.

THE REST OF THE FAMILY

Cross, Self, Semi & Anti Join

Four more variants that turn up constantly in query plans and exam questions — the same Student and Enroll relations throughout.

Cross Join

Student × Enroll

No condition: every Student row is paired with every Enroll row.

```
3 × 3 = 9 tuples
degree 3 + 2 = 5
```

SQL: FROM Student CROSS JOIN Enroll

Self Join

$\rho_{E1}(Emp) \bowtie \rho_{E2}(Emp)$

A relation joined with a renamed copy of itself — needed for hierarchies.

```
Emp(EID, Name, MgrID)
E1.MgrID = E2.EID
```

Answers: who manages whom?

Semi Join

Student ⋉ Enroll

Rows of Student that have at least one partner in Enroll; Enroll's columns are dropped.

```
1901 Rakib 3.80
1903 Nadia 3.92
```

SQL: WHERE EXISTS (SELECT …)

Anti Join

Student ▷ Enroll

The complement: rows of Student with no partner in Enroll at all.

```
1902 Alifa 3.45
```

SQL: WHERE NOT EXISTS (SELECT …)

FROM ALGEBRA TO SQL

Quick Reference

Every operator on the previous slides, with the SQL that implements it. Relational algebra removes duplicates by definition; SQL keeps them unless you say otherwise.

Operation	Relational algebra	SQL equivalent
Union	$R \cup S$	`SELECT … FROM R UNION SELECT … FROM S;`
Intersection	$R \cap S$	`SELECT … FROM R INTERSECT SELECT … FROM S;`
Difference	$R - S$	`SELECT … FROM R EXCEPT SELECT … FROM S;`
Cartesian product	$R \times S$	`SELECT * FROM R CROSS JOIN S;`
Theta join	$R \bowtie_{\theta} S$	`SELECT * FROM R JOIN S ON <condition>;`
Equi join	$R \bowtie_{R.A = S.B} S$	`SELECT * FROM R JOIN S ON R.A = S.B;`
Natural join	$R \bowtie S$	`SELECT * FROM R NATURAL JOIN S;`
Left outer join	$R ⟕ S$	`SELECT * FROM R LEFT OUTER JOIN S ON …;`
Right outer join	$R ⟖ S$	`SELECT * FROM R RIGHT OUTER JOIN S ON …;`
Full outer join	$R ⟗ S$	`SELECT * FROM R FULL OUTER JOIN S ON …;`
Semi join	$R \ltimes S$	`SELECT * FROM R WHERE EXISTS (SELECT 1 FROM S …);`
Anti join	$R \triangleright S$	`SELECT * FROM R WHERE NOT EXISTS (SELECT 1 FROM S …);`